Ohm — Compiler Architecture

A type checker for circuits that happens to emit KiCad

WINTERLABS · PROPRIETARY · V0.1.1 PRE-RELEASE · MASTER @ 37FB632 · 30 AUGUST 2026

> Ohm compiles a declarative description of a *physical artifact* into a netlist, a KiCad project, and a constraint set. That single fact drives every structural decision in the codebase: the centre of gravity is semantic analysis over a graph, not code generation, so Ohm is architecturally closer to a type checker and linter than to a classic lex → parse → codegen compiler. **Checking is the product; emission is mechanical.**

1 What kind of compiler this is

Four consequences follow from the framing above, and they are stated as such in the canonical design document (`docs/blueprint/OHM_COMPILER_DESIGN.md`, read-only):

- The "program" is a **graph** — components, nets, interfaces — not a linear instruction stream. Passes traverse and annotate a graph, not an AST spine alone.
- There is **no runtime**. All meaning resolves at build time, which is what makes hermeticity achievable rather than aspirational.
- The pipeline is therefore **parse → resolve → annotate → check → lower → emit**, with the interesting mass in the middle.
- Every pass is a pure function `pass(ir, ctx) -> (ir', [Diagnostic])`. No pass performs I/O except parse (reading source) and emit (writing files); no pass mutates its input.

The whole thing is roughly 11,200 lines of Python across 46 modules, with one runtime dependency (`lark`). The orchestration — the part that decides what runs in what order and what gets emitted — is **78 lines**, in `ohm/driver.py`.

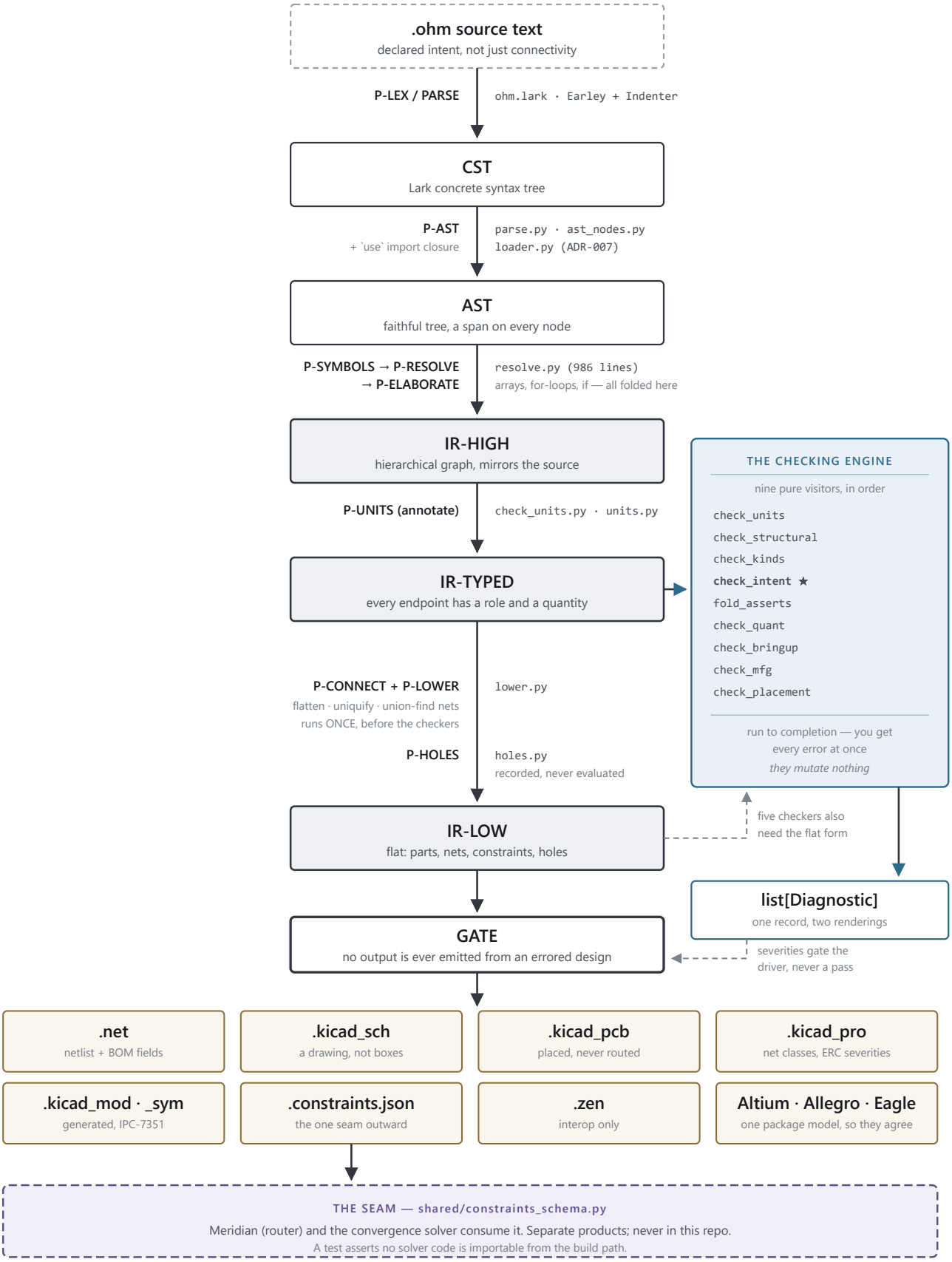


Figure 1 — The compile pipeline. Solid arrows carry data; dashed arrows carry influence. Two details are easy to miss and both matter. **Lowering runs once, before the checkers** — five of the nine need the flat form, so it is computed once and shared rather than recomputed per checker; it is pure, so it is safe even on a partial design that failed to parse. And **the checkers never stop the pipeline**: they only produce diagnostics, and the driver alone decides, at the gate, whether anything is written.

2 The spine: `driver.py`

Eighty lines, and worth reading in full before anything else in the tree. The sequence is load, resolve, annotate units, lower, check, gate, emit. Three properties of that sequence are load-bearing:

- **Parse errors are not fatal.** Resolve and every checker still run on the recovered partial program, so a file with a syntax error still reports its semantic problems. Only lowering and emit are gated. Unresolved names become *poison* bindings that suppress the twenty cascading errors one typo would otherwise produce.
- **Lowering is hoisted above the checkers.** A documented refinement of the blueprint diagram, taken because five checkers need `IRLow` and recomputing it per checker would be wasteful and — worse — could let two checkers reason about different flattenings.
- **The purity invariant is enforced by code.** Under `--debug-passes` the driver re-serializes the design after every checker and raises if one mutated it. "Checkers do not mutate the IR" is a test, not a comment.

3 Three IR levels, and why

A single IR cannot serve parsing, checking, and emission well. Ohm keeps three, each with one job, all of them frozen dataclasses in `ohm/ir.py`.

LEVEL	SHAPE	JOB, AND WHAT IT HOLDS
IR-HIGH	hierarchical graph	Mirrors the source. `Design`, `Module`, `Instance`, raw `~` connections, unresolved quantities. Produced only by the resolver.
IR-TYPED	same shape, annotated	The substrate the checkers read. Tolerances normalized to absolute `tol_lo`/`tol_hi`; a role on every endpoint. Lowering to this level *adds information and keeps shape*.
IR-LOW	flat	The substrate the emitters read. `PartLow`, `NetLow`, `ConstraintLow`, `Hole`. Hierarchy resolved, names uniquified as `<module-path>.<local-name>`. Lowering to this level is *flattening*.

Keeping the two lowerings distinct — annotation, then flattening — is the reason the checker code stays readable. Checkers need hierarchy and types; emitters need flat connectivity; neither has to cope with the other's concerns.

`ir.to_json()` serializes deterministically, and does double duty: it is the golden test oracle and it is the mutation detector. Three modelling choices in that file carry more weight than their size suggests:

- `Pin.must_connect: bool | str | None` — tri-state *plus* net-class values (`"ground"`, `"rail"`). A datasheet saying "RES — connect to GND" means a class of net, not merely some net; a signal net there is as wrong as none. And `None` versus `False` is the difference between nobody having considered the pin and somebody having checked that floating is correct. That distinction is the entire content of diagnostic B011.
- `AssertRec` versus `ObligationRec` — the closed-form / convergence split is enforced *at the IR level*, not by convention. An `assert` folds to a value at build time; an `obligation` becomes a `Hole` and is never evaluated here. Lowering and emit treat the two categorically differently.
- `GenerateSpec` — `package = generate(...)` is frozen as data at elaboration and only becomes geometry in the package-generation pass. Acquisition of pin data happens at authoring time and is frozen into the source; the build path never fetches.

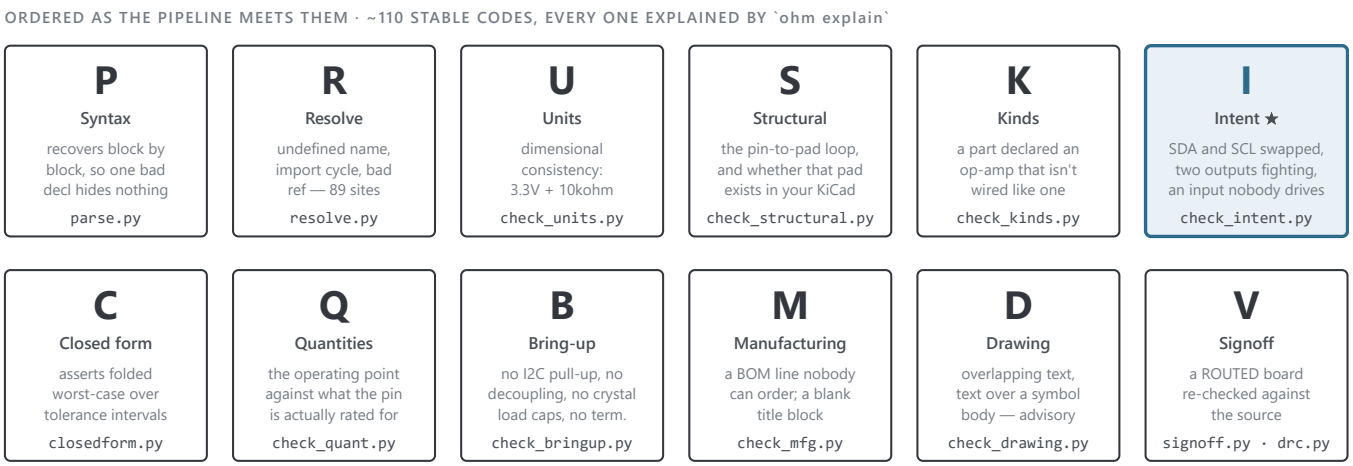


Figure 2 — The diagnostic tiers. The code prefix tells you which pass produced a finding and therefore what kind of thing is wrong. Tiers S through Q are what a compiler normally does. Tiers **B, M, D and V are the product thesis**: the respin-class defects that pass DRC and fail on the bench, because a normal ERC cannot see what a net is *for*. Every one of them can be waived with `waive=`, which leaves an auditable note rather than silence.

4 The checking engine

All nine checkers share one shape — a visitor over IR-TYPED that emits diagnostics and mutates nothing — so they compose and behave consistently. A checker never halts the pipeline itself.

`Diagnostic` is deliberately a product surface rather than an afterthought: one record with two renderings. `machine()` gives `file:line:col: error[I014]: message` for an agent's self-correction loop; `human()` gives the same record as a source excerpt with a caret run, related spans, and a `help:` line. The `EXPLAIN` table in the same module backs `ohm explain CODE`, and a test enforces that it covers every code Ohm can emit.

> WHY THIS MATTERS STRUCTURALLY
>
> Ohm is explicitly designed to be authored by LLM agents (`AGENTS.md`). The stated contract is: iterate on `ohm check --json-diagnostics` until clean, and one diagnostic round typically suffices. That target is what forces diagnostics to be complete (all errors at once), machine-parseable, span-accurate, and individually explainable — and it is why "checkers never abort early" is an architectural rule rather than a nicety.

5 Emission, and the KiCad version policy

`emit_kicad.py` is 2,126 lines, and roughly three quarters of it is the schematic *drawing* engine — the board emitter is about 135 lines, because there is no copper to write. A schematic is treated as a drawing rather than a netlist rendered as boxes: one sheet partitioned into titled dashed blocks per module, a passive that serves one pin of an IC placed against that pin and wired to it, rails as power symbols, and net labels only where a human would actually use one.

One policy shapes the entire backend. **Ohm writes the KiCad 7 format for every target and never bumps the four version stamps**, because KiCad reads *forward* and that is the direction it guarantees. A newer stamp would buy nothing and would cost every older KiCad the ability to open the file.

ARTIFACT	CONSTANT	VALUE	LOCATION
`.kicad_sch`	`_SCH_VERSION`	20230121	`emit_kicad.py:58`
`.kicad_pcb`	`_PCB_VERSION`	20221018	`emit_kicad.py:59`
`.kicad_mod`	`_FP_VERSION`	20221018	`pkggen/footprint.py:9`
`.kicad_sym`	`_SYM_VERSION`	20220914	`pkggen/symbol.py:8`

`--target-kicad {7,8,9,10}` therefore names the *application*, not a serialization; it changes only the footprint pad-id vocabulary and the strictness of one check. The corollary is the genuinely interesting part. Ohm's own *readers* — signoff, `import-symbol`, provenance — face the other way, at files the user's KiCad wrote, so they must be **structural rather than layout-anchored**. A line-anchored parser has already been broken twice by nothing more than a pretty-printer change, and once semantically: KiCad 10 dropped the top-level net table and names nets inline, which made `ohm signoff` report a mismatch on *every pad* of any board that had been opened and saved in KiCad 10.

Placement is real; routing is out of scope

Parts with a `fixed` constraint sit at their declared pose and are emitted locked. Everything else is clustered by module, laid out in wrapped rows, with collisions tested against a live rectangle list. A `near(follower, target, distance)` constraint seats the follower beside its target with **edge-to-edge** distances rounded toward the target, so a declared 2 mm is never silently inflated. Board coordinates use a 0.01 mm quantum rather than the schematic's 1.27 mm grid — that mismatch was the reason a declared `near(x, 2mm)` once came out at 3.27 mm.

The check for overlapping parts runs over the *same* `placement()` function the emitter uses, so the check and the board cannot disagree. There is no router: Ohm emits no copper. Verifying copper that somebody else added is what signoff is for.

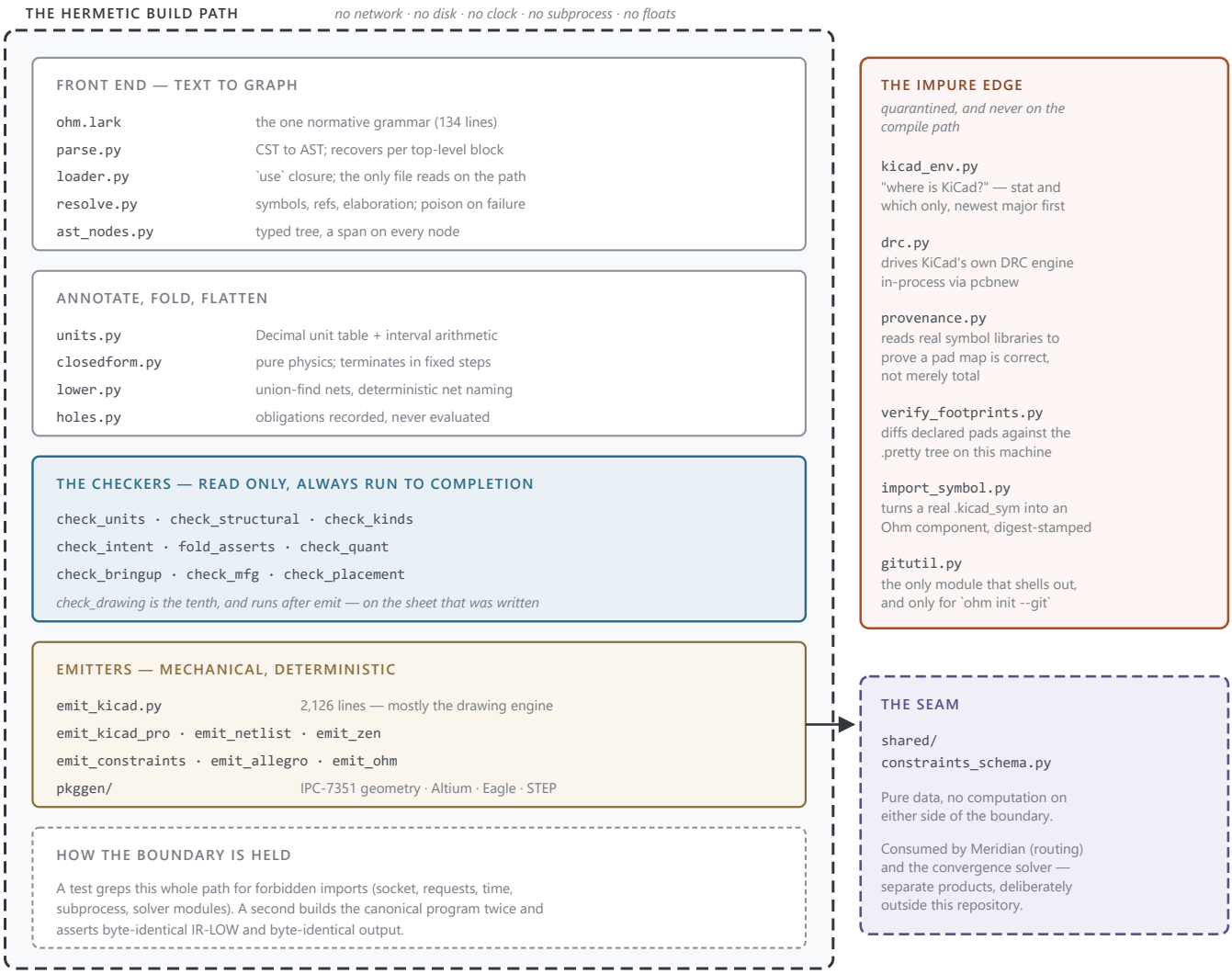


Figure 3 — The module map and the hermetic boundary. The dashed enclosure is not documentation; it is tested. Everything that must touch the world — locating KiCad, running its DRC engine, reading the user's symbol libraries, invoking git — lives outside it, and each of those modules opens with a docstring explaining why it had to. The consequence worth stating plainly: **the compiler cannot produce a different answer on a different machine, on a different day, or with a different network.**

6 What holds it together

Six invariants are named in the design document, and each is held by a mechanism rather than a convention. That pairing is the most transferable thing about this codebase.

INVARIANT	MECHANISM THAT HOLDS IT
Passes are pure; none mutates its input	Debug mode re-serializes the design after every checker and raises on any difference
The three IR levels stay distinct	Checkers take `Design`; emitters take `IRLow`. The type signatures are the fence
Checkers never halt the pipeline	Only the driver reads severities; a checker returns a list and nothing else
No output from an errored design	A single `if emit and result.ok` before anything is written
Closed form and convergence stay separate	Enforced in the IR type: `AssertRec` folds, `ObligationRec` becomes a `Hole`
Deterministic and hermetic	Double-build byte-identity test; forbidden-import grep over the build path

Above those sit two gates that do the heaviest lifting, and both work by refusing to trust Ohm's own opinion of its output:

- **Connectivity is proved, exactly.** The round-trip test makes *KiCad itself* re-read the emitted schematic and extract the net graph, then compares it to Ohm's own netlist. A wire that misses its pin splits a net and the gate fails. This is what made every layout rewrite in the drawing engine safe to attempt.
- **Read-back is proved too.** A second gate runs an emitted board through `kicad-cli pcb upgrade` — KiCad rewriting the file, exactly as it does the moment anyone opens and saves — and asserts signoff reaches the same verdict. Everything else tests what Ohm *writes*; this is the only gate on what Ohm *reads*.

Continuous integration runs the KiCad-dependent suite *inside* the official `kicad/kicad` containers at 8.0.9, 9.0.9 and 10.0.5, so the binary and its libraries are the same version by construction, and an environment variable converts any KiCad-shaped skip into a failure. Measured: 342 passed, 0 skipped across the matrix.

7 Open items

Two things a reader of this architecture should know are unfinished rather than settled.

- **ADR-013, spatial stackup, is still *proposed*.** It would add a `stackup` declaration and a layout tier — the spatial analogue of the bring-up tier, aimed at return-path discontinuity. It is the one open architectural direction, and nothing in the tree implements it yet.
- **A dangling diagnostic reference.** `trace_width.py:35` states that the project emitter reports code M013 with the numbers it used. M013 does not exist anywhere in the repository, so the copper-weight and temperature-rise assumptions (1 oz, 10 °C) are baked into every computed track width and never surfaced to the user. It is a small gap in an otherwise strict "declare, don't assume" posture.

> STATED SCOPE, WORTH REPEATING
>
> Ohm's own README is explicit that **no Ohm-designed board has been manufactured yet**. The bring-up checks are tested, not field-proven. That honesty is itself an architectural commitment — the same one that keeps ratings undeclared rather than invented, and keeps template pinouts as comments rather than as components that would netlist wrong.